

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосунків»

Практична робота №7

Маніпуляція DOM за допомогою JavaScript

Виконав:

Студент II курсу

групи ІІІ-56

Гончар І.В.

Перевірив:

Ковтунець О.В.

1. Мета роботи:

Навчитися працювати з деревом DOM засобами чистого JavaScript: вибирати елементи сторінки через `querySelector`, змінювати їхній вміст, атрибути й класи, а також динамічно створювати та видаляти елементи (медитами `createElement`, `append`, `remove`) для виведення списку даних масиву замість статичної розмітки-заглушки.

2. Хід виконання роботи

Крок 1–3. Підготовка DOM-контейнера та видалення заглушки:

У файл `index.html` було додано ідентифікатор `id="games-list"` для контейнера карток та `id="games-count"` для лічильника. У файлі `js/script.js` за допомогою методу `document.querySelector` та властивості `.remove()` програмно видалено статичну картку-зглушку. Фрагмент реалізації функції рендеру та роботи з DOM наведено нижче (рис. 1).

```
js> script.js >...
1 // 1. Додаємо картинку та опис у масив
2 const games = [
3   { title: 'Каркасон', minPlayers: 2, maxPlayers: 5, img: 'assets/img/carcassonne.jpg', desc: 'Класична гра на вибудовування міст та дор.' },
4   { title: 'Манчкин', minPlayers: 3, maxPlayers: 6, img: 'assets/img/munchkin.jpg', desc: 'Весела карткова гра про зачистку підземель.' },
5   { title: 'Діксир', minPlayers: 3, maxPlayers: 8, img: 'assets/img/dixit.jpg', desc: 'Гра на асоціації з неймовірними ілюстраціями.' }
6 ];
7
8 function fitsPlayers(game, players) {
9   return players >= game.minPlayers && players <= game.maxPlayers;
10 }
11
12 const listContainer = document.querySelector("#games-list");
13 const gamesCount = document.querySelector("#games-count");
14
15 // Очищаємо статичні картки-зглушки при старті
16 if (listContainer) listContainer.innerHTML = '';
17
18 function renderGames(gamesArray) {
19   listContainer.innerHTML = '';
20
21   gamesArray.forEach(game => {
22     const card = document.createElement('article');
23     card.classList.add('card'); // Той клас із CSS
24
25     const title = document.createElement('h3');
26     title.textContent = game.title;
27
28     // Відновлюємо новий бейдж
29     const badge = document.createElement('span');
30     badge.classList.add('badge');
31     badge.textContent = `${game.minPlayers}-${game.maxPlayers} гравців`;
32
33     // Відновлюємо картинку
34     const image = document.createElement('img');
35     image.src = game.img;
36     image.alt = `Обкладинка гри ${game.title}`;
37     // Додає клас картинці, якщо він був у твоєму CSS (наприклад, .card-img)
38     // Image.classList.add('card-img');
39
40     // Відновлюємо опис
41     const desc = document.createElement('p');
42     desc.textContent = game.desc;
43
44     // Атрибут та клас за умовами 7-1 лаби
45     card.dataset.players = `${game.minPlayers}-${game.maxPlayers}`;
46     if (fitsPlayers(game, 4)) {
47       card.classList.add('fits');
48     }
49
50     // Вкладаємо всі елементи в картку в потрібному порядку
51     card.append(title, badge, image, desc);
52     listContainer.append(card);
53   });
54
55   if (gamesCount) gamesCount.textContent = gamesArray.length;
56 }
57
58 renderGames(games);
```

Рис. 1 — Реалізація функції динамічного рендеру у файлі `script.js`

Крок 4–7. Написання функції динамічного рендеру та додавання атрибутів:

Оголошено функцію `renderGames(gamesArray)`, яка через цикл `forEach` створює теги `<article>` та дочірні елементи. Для кожного елемента через об'єкт `dataset` встановлено атрибут `data-players`, а за допомогою перевірки `fitsPlayers(game, 4)` умовно додано клас `.fits`. Готові вузли додано до контейнера методом `.append()`.

Крок 8–9. Виклик рендеру та оновлення підсумку:

Функцію рендеру викликано під час завантаження сторінки, оновлено лічильник ігор. Для перевірки візуального відображення та кросбраузерності проєкт протестовано у двох браузерах — Google Chrome та Opera GX. Результати відображення каталогу наведено нижче (рис. 2, 3).

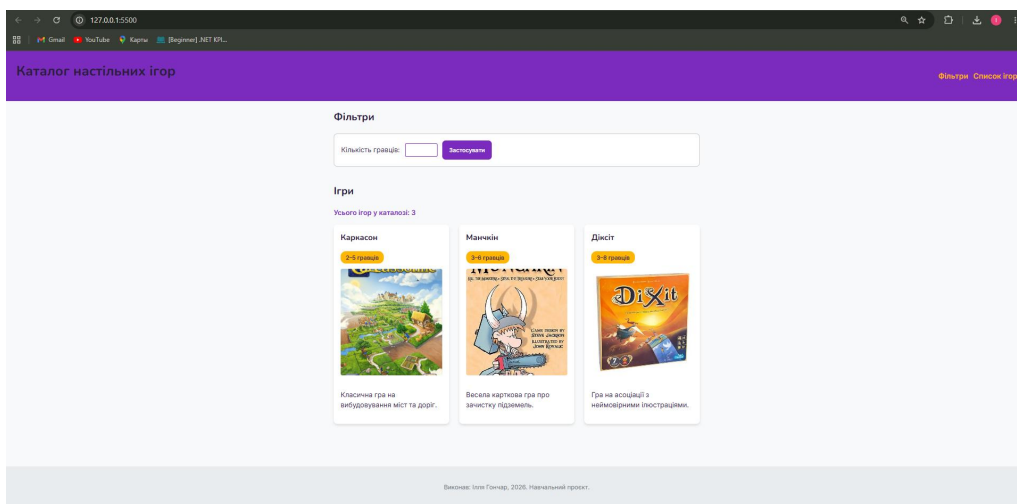


Рис. 2 — Динамічний рендер каталогу у браузері Google Chrome

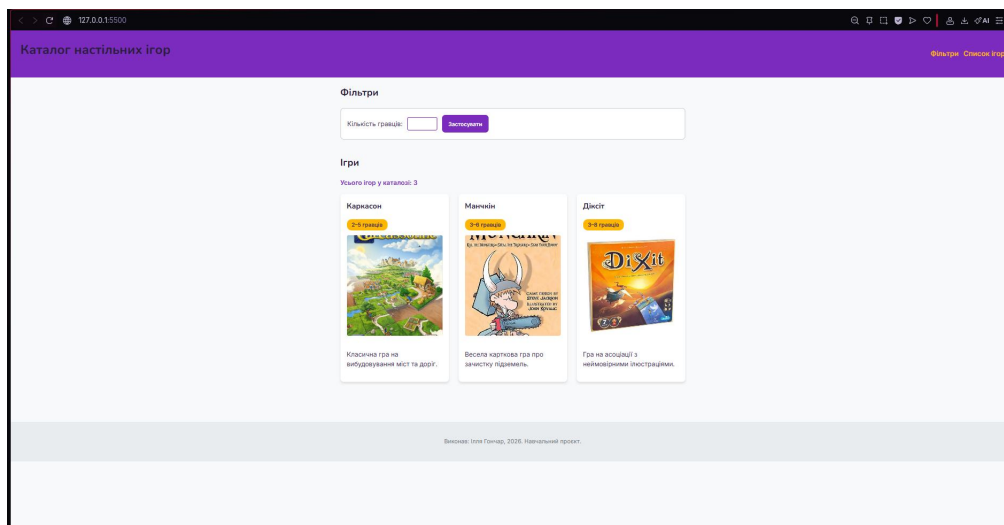


Рис. 3 — Перевірка відображення каталогу у браузері Opera GX

Крок 10. Перевірка в DevTools:

За допомогою інструментів розробника перевірено структуру згенерованого DOM-дерева, наявність тегів `<article>`, атрибутів `data-players` та відсутність помилок у консолі. Перегляд дерева елементів зафіксовано нижче (рис. 4).

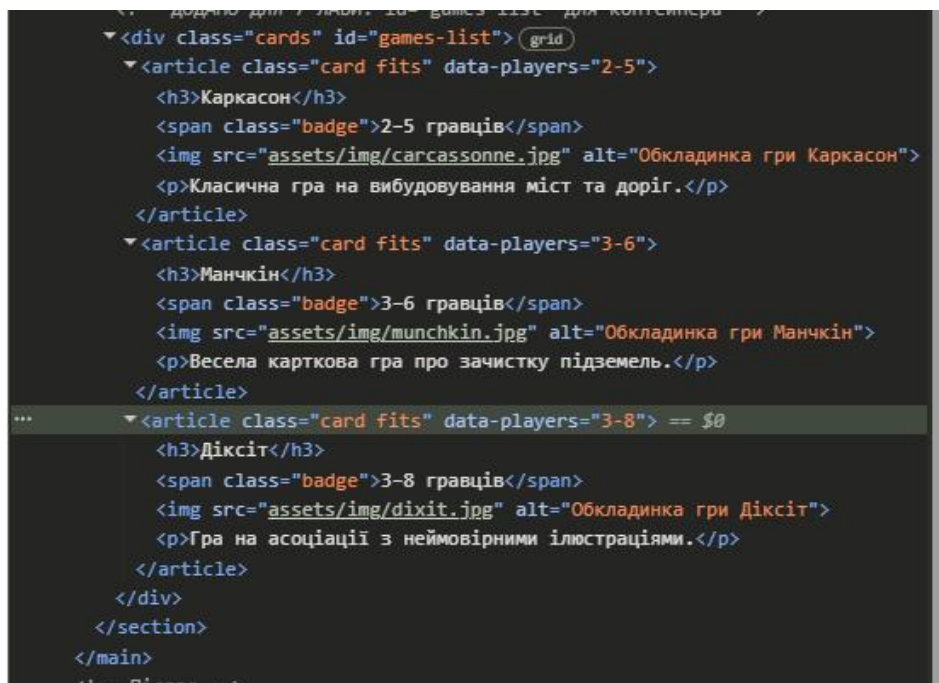


Рис. 4 — Перевірка згенерованої структури DOM-дерева у вкладці Elements (F12)

Крок 11. Збереження та відправка:

Усі зміни зафіксовано за допомогою системи контролю версій Git та відправлено до віддаленого репозиторію на GitHub. Фінальний стан репозиторію наведено нижче (рис. 5).

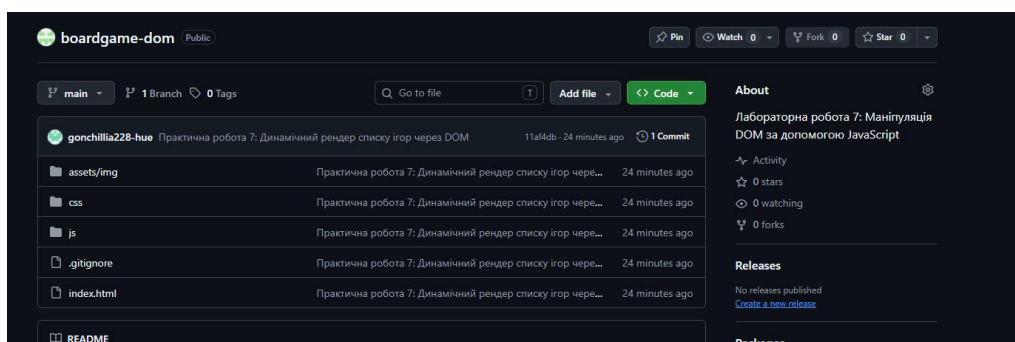


Рис. 5 — Оновлений проект у віддаленому репозиторії на GitHub

Посилання на репозиторій:

<https://github.com/Bitshkiper218/boardgame-dom>

Висновок:

Під час виконання практичної роботи було успішно засвоєно навички маніпуляції деревом DOM засобами чистого JavaScript. Статичну розмітку каталогу настільних ігор повністю замінено на динамічну генерацію карток із масиву даних за допомогою методів `createElement`, `append` та `remove`. Програмно видалено статичну заглушку, реалізовано створення тегів `<article>` для кожної гри, а також заповнено їхній внутрішній вміст (`h3`, `span`, `img`, `p`). Для елементів каталогу налаштовано роботу з довільними атрибутами через об'єкт `dataset` (зокрема `data-players`) та динамічним керуванням класами через `classList` за умовами завдання. Додатково оновлено текстовий вміст лічильника кількості ігор на основі довжини масиву даних. Працездатність коду перевірена за допомогою інструментів розробника (DevTools) у браузерях Google Chrome та Opera GX, де підтверджено правильність структури згенерованого DOM-дерева без помилок у консолі. Фінальний результат роботи та проєкт успішно збережено за допомогою системи контролю версій Git та опубліковано у віддаленому репозиторії на GitHub.